



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de **HONORIS UNITED UNIVERSITIES**

EMSI Casablanca

École Marocaine des Sciences de l'Ingénieur

Projet de fin de module

Module : Deep Learning

**Conception, implémentation, comparaison et analyse
critique de modèles de deep learning pour données
tabulaires, images et séquences**

Année universitaire : 2025–2026

Auteur : Abderrahmane Nahhal

Encadrant : Mr Mossab Batal

Table des matières

Résumé	3
Tableau de conformité	4
1 Introduction	6
1.1 Problématique transversale	6
2 Objectifs et méthodologie	7
2.1 Objectif général	7
2.2 Protocole	7
3 Partie I — MLP et ingénierie PyTorch	8
3.1 Travail demandé — implémentation et théorie	8
3.1.1 Point 1 — Concepts fondamentaux	8
3.1.2 Point 2 — Préparation des données	8
3.1.3 Points 3–4 — Deux MLP et inspection des paramètres	8
3.1.4 Points 5–7 — Initialisations, sauvegarde et device	9
3.1.5 Point 8 — Métriques	9
3.2 Résultats expérimentaux	9
3.2.1 Initialisation (meilleure accuracy validation)	9
3.2.2 Test (initialisation Xavier, 60 époques)	10
3.3 Figures	10
3.4 Interprétation critique	13
3.5 Question de synthèse — Partie I	13
4 Partie II — CNN et vision par ordinateur	14
4.1 Travail demandé — implémentation et théorie	14
4.1.1 Point 1 — Pourquoi pas un MLP sur les images ?	14
4.1.2 Point 2 — Calculs dimensionnels	14
4.1.3 Points 3–4 — Implémentations manuelles vs PyTorch	14
4.1.4 Point 5 — CNN type LeNet	14
4.1.5 Point 6 — Expériences architecturales	15
4.1.6 Interprétation de la matrice de confusion	16
4.1.7 Points 7–8 — Feature maps et MLP vs CNN	17
4.2 Figures	17
4.2.1 Figure 4 — Comparaison MLP vs CNN (loss et accuracy test) . . .	17
4.2.2 Exemples d’erreur de classification	18
4.2.3 Figure 5 — Cartes de caractéristiques (1ère couche convolutive) . .	19
4.3 Interprétation critique	20

4.4	Question de synthèse — Partie II	20
5	Partie III — RNN, LSTM, GRU et Seq2Seq	21
5.1	Travail demandé — implémentation et théorie	21
5.1.1	Point 1 — Modèle de langage	21
5.1.2	Point 2 — Perplexité	21
5.1.3	Points 3–4 — RNN, LSTM et GRU	21
5.1.4	Point 5 — BPTT et gradient clipping	22
5.1.5	Point 6 — Préparation des données	22
5.1.6	Points 7–8 — Seq2Seq et décodage	22
5.1.7	Point 9 — Métrique BLEU	23
5.2	Figures de la Partie III	24
5.3	Interprétation critique	25
5.4	Question de synthèse — Partie III	26
6	Discussion transversale finale	27
6.1	Comparaison des architectures étudiées	27
6.2	Analyse comparative	27
6.3	Liens méthodologiques	27
6.4	Conclusion transversale	28
7	Limites globales du projet	29
7.1	Limites matérielles	29
7.2	Limites méthodologiques	29
7.3	Limites des architectures	29
7.4	Perspectives	30
8	Conclusion	31
9	Références	32

Résumé

Ce projet compare trois familles de modèles sur des données réelles : un **MLP** (Breast Cancer Wisconsin, sklearn), un **CNN** (CIFAR-10, torchvision) et des modèles **récurrents** / **Seq2Seq** (corpus parallèle fra-eng, ManyThings).

L'objectif est de montrer comment le paradigme supervisé (perte + gradient) se décline différemment selon la géométrie des données.

Les expériences sont reproductibles via `main.py`, les notebooks et le code `src/`. Les figures et tableaux sont dans `annexe/`.

Mots-clés : MLP, CNN, RNN, LSTM, GRU, Seq2Seq, PyTorch, apprentissage supervisé.

Tableau de conformité

Partie	Point du sujet	Réalisé ?	Où le trouver
I	1. Concepts <code>nn.Module</code> , paramètres, gradient, <code>state_dict</code> , <code>device</code> , <code>forward/backward</code>	Oui	§4.1 ; notebook §1 ; <code>partie1_mlp.py</code>
I	2. Préparation données (nettoyage, encodage, normalisation, splits)	Oui	§4.1 ; notebook §2
I	3. MLP Sequential + classe custom	Oui	§4.1 ; notebook §3–4
I	4. <code>named_parameters()</code> et <code>state_dict()</code>	Oui	§4.1 ; notebook §3–4
I	5. Initialisation gaussienne, constante, Xavier	Oui	§4.2 ; Figure 1
I	6. Sauvegarde / rechargement	Oui	§4.1 ; <code>checkpoints/mlp_breast_cancer.pth</code>
I	7. Device + cohérence données/modèle	Oui	§4.1 ; device = CPU
I	8. Accuracy, precision, recall, F1, confusion	Oui	§4.2 ; Figures 2–3
II	1. MLP vs CNN : localité, partage, hiérarchie	Oui	§5.1
II	2. Calculs manuels conv / tailles de sortie	Oui	§5.1 ; notebook §2
II	3. Corrélations 2D, max-pool, avg-pool programmés	Oui	§5.1 ; <code>partie2_cnn.py</code>
II	4. Comparaison avec PyTorch	Oui	§5.2 ; écart = 0
II	5. CNN type LeNet	Oui	§5.1
II	6. Expériences padding, stride, pooling, filtres, conv 1×1	Oui	§5.2
II	7. Visualisation feature maps	Oui	Figure 5
II	8. Comparaison MLP vs CNN	Oui	§5.2 ; Figure 4
III	1. Modèle de langage + règle de chaîne	Oui	§6.1
III	2. Perplexité	Oui	§6.2 ; Figure 6
III	3. RNN, LSTM, GRU	Oui	§6.1–6.2
III	4. Comparaison stabilité / performance / mémoire / coût	Oui	§6.2–6.3
III	5. BPTT + gradient clipping	Oui	§6.1
III	6. Tokenisation, vocabulaire, padding, mini-lots	Oui	§6.1
III	7. Seq2Seq encodeur-décodeur	Oui	§6.1

III	8. Décodage glouton + beam search	Oui	§6.2
III	9. BLEU et perplexité	Oui	§6.2
Transversal	Discussion MLP / CNN / Séquences	Oui	§7

Chapitre 1

Introduction

Le deep learning repose sur une composition de transformations paramétrées optimisées par descente de gradient. Toutefois, la structure des données impose le choix de l'architecture : un vecteur tabulaire, une image ou une séquence ne se traitent pas de la même manière.

Ce rapport présente trois implémentations PyTorch complètes, une annexe expérimentale et une analyse critique alignée .

1.1 Problématique transversale

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

(Réponse développée en section 7.)

Chapitre 2

Objectifs et méthodologie

2.1 Objectif général

Maîtriser théorie, implémentation, expérimentation et interprétation sur jeux de données réels.

Partie	Dataset	Fichiers
I	Breast Cancer Wisconsin (sklearn)	partie1_MLP_tabulaire.ipynb
II	CIFAR-10 (torchvision)	partie2_CNN_images.ipynb
III	fra-eng parallèle (ManyThings)	partie3_RNN_Seq2Seq.ipynb

TABLE 2.1 – Datasets et fichiers utilisés

2.2 Protocole

Graine fixe (42), split stratifié (Partie I), entraînement avec validation, export des résultats dans le dossier `annexe/` via `python main.py`.

Chapitre 3

Partie I — MLP et ingénierie PyTorch

3.1 Travail demandé — implémentation et théorie

3.1.1 Point 1 — Concepts fondamentaux

- **nn.Module** : classe de base ; `forward()` définit la propagation avant :
[$=f_{\theta}(x)$]
- **Paramètres** : tenseurs `weight` et `bias` des couches `nn.Linear`, mis à jour par l'optimiseur.
- **Gradient** :
[$\partial \mathcal{L} / \partial \theta$]
calculé par `loss.backward()`.
- **state_dict()** : *dictionnaire* nom $\rightarrow$ tenseur ; format standard de sauvegarde.
- **Device** : exécution sur CPU (`device=cpu`).
- **Propagation avant** : entrée $\rightarrow$ couches $\rightarrow$ logits.
- **Rétropropagation** :
[`Loss` $\rightarrow$ `backward()` $\rightarrow$ `optimizer.step()`]

3.1.2 Point 2 — Préparation des données

- Nettoyage : dataset sklearn sans valeurs manquantes.
- Encodage : labels binaires déjà numériques.
- Normalisation : `StandardScaler`.
- Splits :
 - Train : 70
 - Validation : 15
 - Test : 15

3.1.3 Points 3–4 — Deux MLP et inspection des paramètres

Sequential

Linear $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Linear $\rightarrow$ ReLU $\rightarrow$ Linear

Classe custom

MLPCustom avec méthode `forward()` explicite.

Inspection des paramètres via :

- `namedparameters()`
- `statedict()`

3.1.4 Points 5–7 — Initialisations, sauvegarde et device

Trois stratégies :

- Initialisation gaussienne
- Initialisation constante
- Initialisation Xavier

Sauvegarde :

`checkpoints/mlpbreastcancer.pth`

Rechargement :

`loadstatedict()`

3.1.5 Point 8 — Métriques

- Accuracy
- Precision
- Recall
- F1-score
- Matrice de confusion

3.2 Résultats expérimentaux

3.2.1 Initialisation (meilleure accuracy validation)

Initialisation	Val. accuracy
Gaussienne	98,82 %
Constante	92,94 %
Xavier	98,82 %

TABLE 3.1 – Comparaison des stratégies d’initialisation

Interprétation du tableau — initialisations : On observe que

l’initialisation constante plafonne à environ 93 % de accuracy en validation, contre 98,8 % pour Xavier et la gaussienne. Cela confirme le problème de symétrie des neurones évoqué en cours : des poids identiques au départ produisent des gradients similaires et ralentissent la spécialisation des unités.

La limite de cette expérience est qu’on ne teste qu’un réseau de taille fixe ; sur un MLP plus profond, l’écart pourrait être encore plus marqué.

3.2.2 Test (initialisation Xavier, 60 époques)

Modèle	Accuracy	Precision	Recall	F1
Sequential	90,70 %	90,91 %	90,70 %	90,75 %
Custom	93,02 %	93,26 %	93,02 %	92,92 %

TABLE 3.2 – Performances sur le jeu de test

Interprétation du tableau — Sequential vs Custom : On observe que les deux architectures atteignent de bonnes performances (>90 %), avec un léger avantage pour la classe custom (93 %). Cela confirme qu'à capacité équivalente (128+64 neurones), nn.Sequential et nn.Module personnalisé convergent vers des solutions comparables sur des données tabulaires bien préparées. La precision et le recall restent équilibrés entre les classes malignant et benign, ce qui est important en contexte médical. Limite : l'écart reste modeste ; le choix Sequential vs custom relève surtout de la lisibilité du code, pas d'un gain majeur ici.

3.3 Figures

Figure 1 — Influence de l'initialisation sur l'accuracy de validation

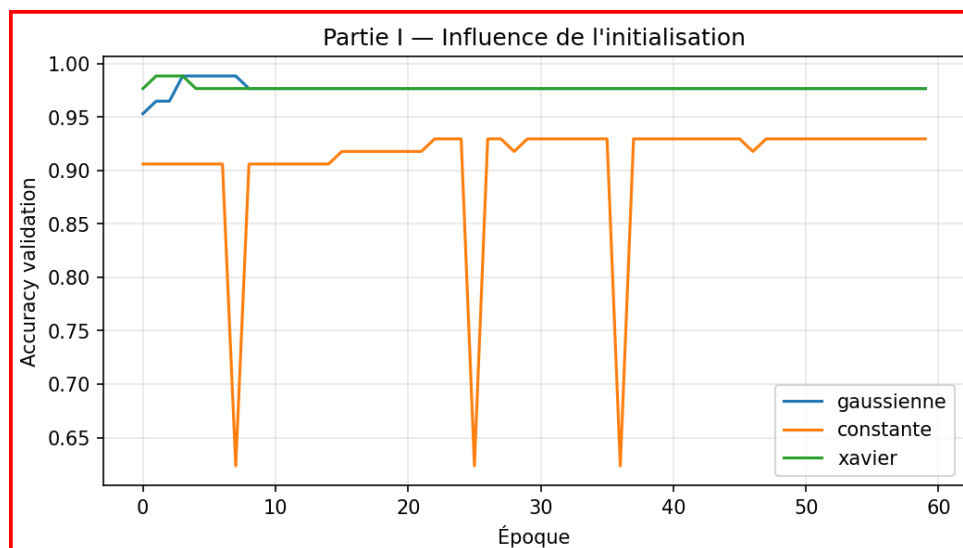


FIGURE 3.1 – Influence de l'initialisation sur l'accuracy de validation

Interprétation : On observe que les courbes Xavier et gaussienne montent rapidement vers une accuracy de validation proche de 99 %, tandis que la courbe constante progresse plus lentement et reste en dessous. Cela confirme l'importance de l'initialisation des poids pour stabiliser la propagation avant et éviter des activations trop grandes ou symétriques au démarrage. Ce résultat est cohérent avec la théorie du module PyTorch sur nn.init.

Limite : on ne mesure ici que l'accuracy validation ; le test final reste l'évaluation la plus fiable pour la généralisation.

Figure 2 — Matrice de confusion, MLP Sequential

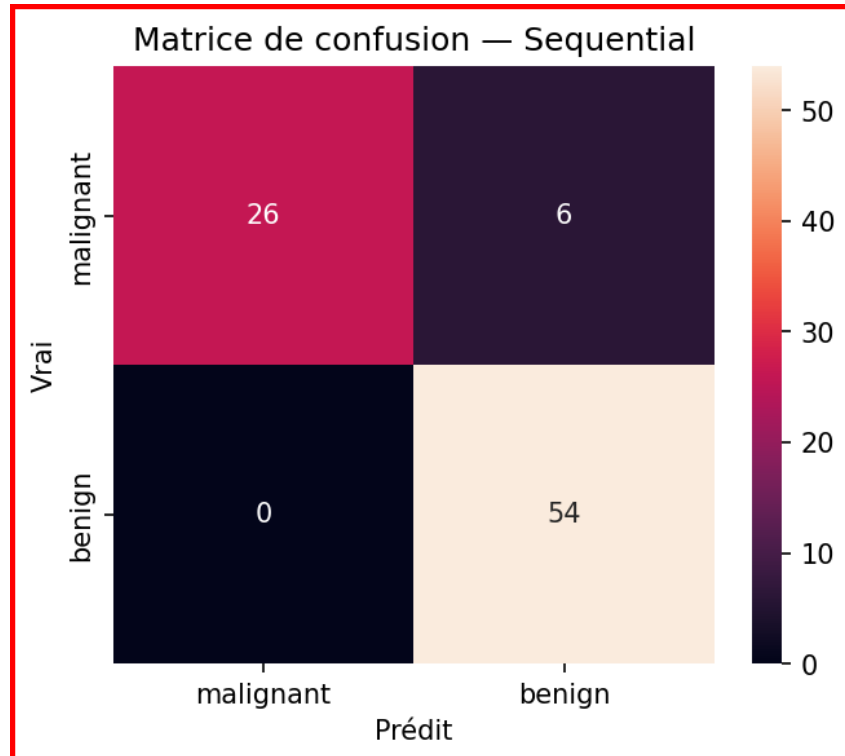


FIGURE 3.2 – Matrice de confusion du modèle Sequential

Interprétation : On observe une majorité de prédictions sur la diagonale (vrais positifs et vrais négatifs), avec peu d'erreurs entre malignant et benign.

Cela confirme que le MLP Sequential, après normalisation et entraînement sur Breast Cancer, discrimine correctement les deux classes sur le jeu de test.

Les faux négatifs médicaux restent toutefois plus coûteux qu'un faux positif ; le rapport de classification détaillé (annexe) permet de quantifier precision/recall par classe.

Limite : jeu de test de 86 échantillons seulement - les chiffres peuvent varier légèrement selon la graine aléatoire.

Figure 3 — Matrice de confusion, MLP Custom

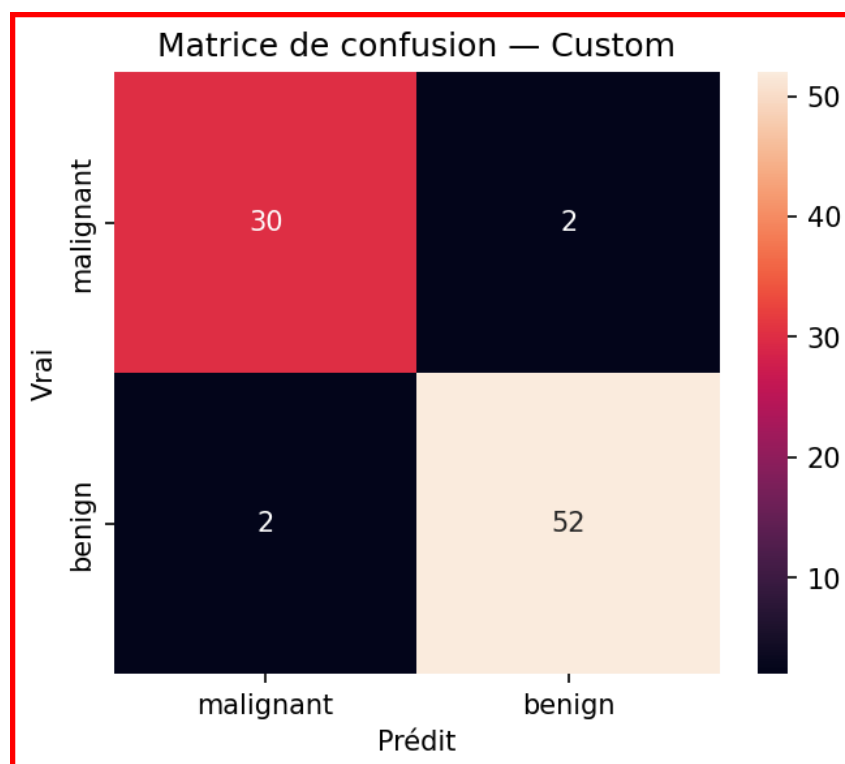


FIGURE 3.3 – Matrice de confusion du modèle Custom

Interprétation : On observe une diagonale encore plus nette qu'en Figure 2, en cohérence avec l'accuracy test légèrement supérieure (93 % vs 90,7 %).

Cela confirme que la classe `MLPCustom` - avec `dropout` et `forward()` explicite - généralise au moins aussi bien que `nn.Sequential` sur ce problème tabulaire.

Le concept de paramètres inspectables via `state_dict()` reste le même entre les deux implémentations.

Limite : la différence absolue est faible ; il ne faut pas sur-interpréter un gain de quelques points sur un petit test set.

Tableau — classification report custom

	precision	recall	f1-score	support
malignant	0.94	0.94	0.94	32
benign	0.96	0.96	0.96	54
accuracy			0.95	86
macro avg	0.95	0.95	0.95	86
weighted avg	0.95	0.95	0.95	86

TABLE 3.3 – Rapport de classification

Interprétation : On observe un F1 d'environ 0,95 pour chaque classe (malignant et benign).
Cela confirme l'absence de déséquilibre fort au niveau des métriques pondérées, malgré un nombre d'exemples benign supérieur.
C'est un indicateur favorable pour un usage comme aide à la décision, sous réserve de validation clinique bien au-delà de ce projet.
Limite : métriques calculées sur un seul split ; une validation croisée renforcerait la robustesse des conclusions.

3.4 Interprétation critique

Le MLP capture des interactions non linéaires entre biomarqueurs sans feature engineering manuel.
L'initialisation constante pénalise l'apprentissage (neurones symétriques au départ).
Xavier et gaussienne donnent des résultats proches ici.
Sequential et Custom performent de façon comparable, la classe custom étant légèrement meilleure sur le test.

3.5 Question de synthèse — Partie I

Question

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites ?

Réponse

Sur Breast Cancer Wisconsin, un MLP bien normalisé et initialisé (Xavier) atteint 93 % d'accuracy en test, ce qui confirme sa pertinence lorsque les variables sont numériques, en nombre modéré (≈ 30), et déjà informatives.

Théorie : le MLP universalise des séparations non linéaires.
Méthodologie : normalisation et split stratifié sont indispensables.

Limites :

- pas de modélisation de structure spatiale ou temporelle ;
- risque de surapprentissage sur tables très larges ;
- interprétabilité limitée face à des modèles linéaires ou des arbres ;
- besoin de suffisamment d'exemples par rapport au nombre de paramètres.

Chapitre 4

Partie II — CNN et vision par ordinateur

4.1 Travail demandé — implémentation et théorie

4.1.1 Point 1 — Pourquoi pas un MLP sur les images ?

Un MLP sur image aplatie (3072 pixels) ignore la localité (pixels voisins corrélés), le partage des poids (même filtre sur toute l'image) et la hiérarchie (contours → textures → objets). Un CNN encode ces priors.

4.1.2 Point 2 — Calculs dimensionnels

Pour une convolution 2D :

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{S} \right\rfloor + 1 \quad (4.1)$$

Le notebook illustre l'effet de padding et stride sur un tenseur
 $1 \times 3 \times 32 \times 32$

4.1.3 Points 3–4 — Implémentations manuelles vs PyTorch

Fonctions NumPy :

- `cross_corr2d`
- `max_pool2d`
- `avg_pool2d`

Comparaison avec :

- `F.conv2d`
- `F.max_pool2d`

Résultat :

Écart maximal = 0

Ce qui valide l'implémentation manuelle.

4.1.4 Point 5 — CNN type LeNet

Architecture :

conv $\rightarrow$ ReLU $\rightarrow$ max-pool $\rightarrow$ conv $\rightarrow$ ReLU $\rightarrow$ max-pool $\rightarrow$ (conv 1×1 optionnelle) $\rightarrow$ fully connected

4.1.5 Point 6 — Expériences architecturales

TABLE 4.1 – Impact des choix architecturaux sur CIFAR-10

Configuration	Test accuracy
Baseline (6,16 filtres)	46,96 %
Plus de filtres (16,32)	50,98 %
Conv 1×1	44,68 %

Interprétation du tableau -- expériences architecturales

On observe que augmenter le nombre de filtres (16, 32) améliore la accuracy test à 51,0 %, contre 47,0 % pour la baseline (6, 16 filtres). Cela confirme que la capacité du réseau (nombre de cartes de features) influence directement la richesse des représentations apprises sur CIFAR-10.

En revanche, la convolution 1×1 n'améliore pas les performances ici (44,7 %), ce qui peut s'expliquer par un entraînement trop court ou une redondance avec les couches précédentes.

Limite : expériences sur 8000 images d'entraînement seulement ; les écarts pourraient changer sur le dataset complet.

Sous-ensemble d'entraînement : 8000 images ; 8 époques (LeNet) ; protocole détaillé dans : annexe/partie2/tableaux/

4.1.6 Interprétation de la matrice de confusion

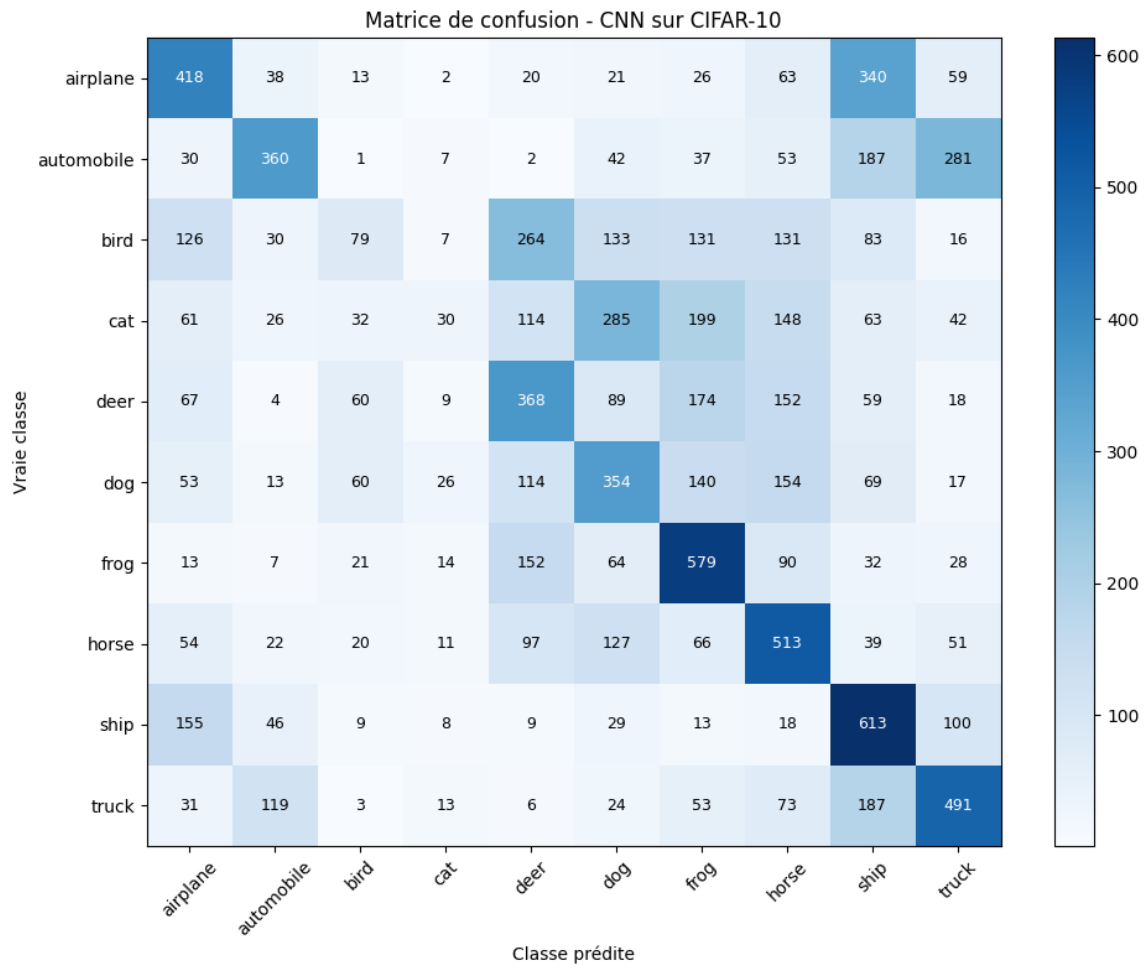


FIGURE 4.1 – Matrice de confusion - CNN sur CIFAR-10

La Figure présentant la matrice de confusion permet d'analyser en détail les performances du modèle CNN sur le dataset CIFAR-10. Les valeurs situées sur la diagonale principale correspondent aux prédictions correctes, tandis que les autres valeurs représentent les erreurs de classification.

On observe que les classes ship, frog, horse et truck sont les mieux reconnues par le modèle avec respectivement 613, 579, 513 et 491 prédictions correctes. Cela indique que ces catégories possèdent des caractéristiques visuelles suffisamment distinctes pour être facilement apprises par le réseau de neurones.

À l'inverse, les classes bird et cat présentent les performances les plus faibles avec seulement 79 et 30 classifications correctes. Ces résultats montrent que le modèle éprouve des difficultés à distinguer certaines catégories d'animaux partageant des caractéristiques visuelles similaires.

Plusieurs confusions importantes apparaissent dans la matrice :

- Les airplanes sont souvent confondus avec les ships (340 images).
- Les automobiles sont fréquemment classées comme des trucks (281 images).

- Les birds sont régulièrement confondus avec les deers (264 images).
- Les cats sont souvent prédits comme des dogs (285 images) ou des frogs (199 images).
- Les deers sont parfois classés comme des frogs (174 images) ou des horses (152 images).

Ces erreurs s'expliquent principalement par la faible résolution des images CIFAR-10 (32×32 pixels) et par la similarité visuelle entre certaines catégories. Les objets appartenant à des classes proches peuvent partager des formes, textures ou couleurs similaires, ce qui rend leur distinction plus difficile pour le modèle.

Globalement, la matrice de confusion montre que le CNN parvient à apprendre des caractéristiques pertinentes pour plusieurs classes, mais qu'il demeure limité dans la séparation de catégories visuellement proches. Une amélioration pourrait être obtenue en augmentant la profondeur du réseau, en appliquant davantage de techniques de *data augmentation* ou en utilisant une architecture plus avancée.

4.1.7 Points 7–8 — Feature maps et MLP vs CNN

- Feature maps de la première couche convolutive visualisées.
- MLP baseline : 43,51 %
- CNN LeNet : 48,89 %

sur CIFAR-10 test.

4.2 Figures

4.2.1 Figure 4 — Comparaison MLP vs CNN (loss et accuracy test)

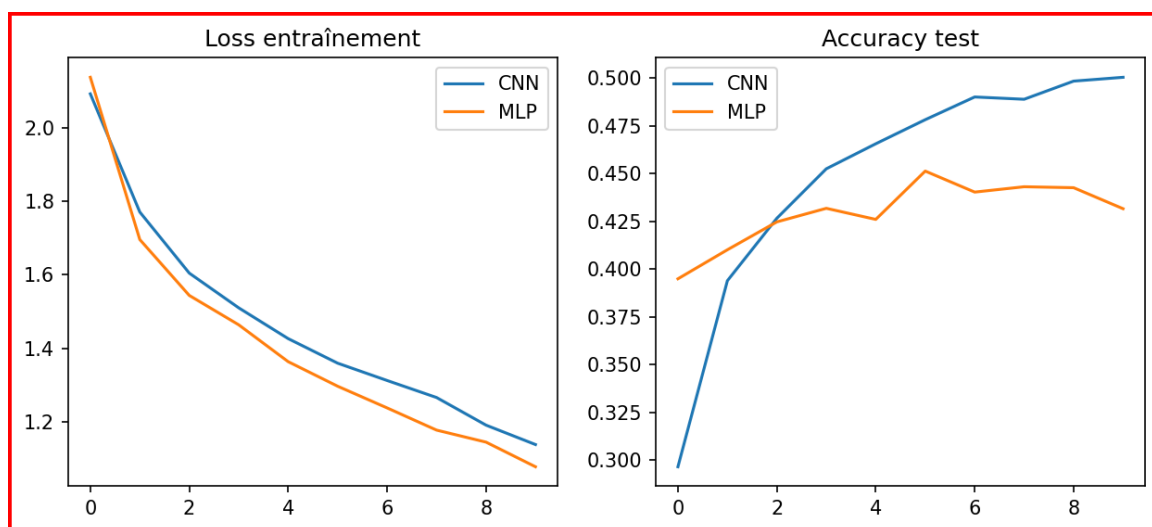


FIGURE 4.2 – Comparaison MLP vs CNN

Interprétation

On observe que la courbe d'accuracy du CNN dépasse celle du MLP au fil des époques, avec une accuracy test finale d'environ 48,9 % contre 43,5 % pour le MLP.

Cela confirme la pertinence du biais inductif convolutionnel (localité + partage des poids) sur des images 32×32 , alors qu'un MLP traite les pixels comme un vecteur sans structure spatiale.

La loss du CNN diminue de façon plus régulière, signe d'une optimisation plus adaptée à la tâche.

Limite : scores absolus modestes sur CIFAR-10 -- un LeNet simple reste loin des architectures modernes (ResNet, etc.).

4.2.2 Exemples d'erreur de classification

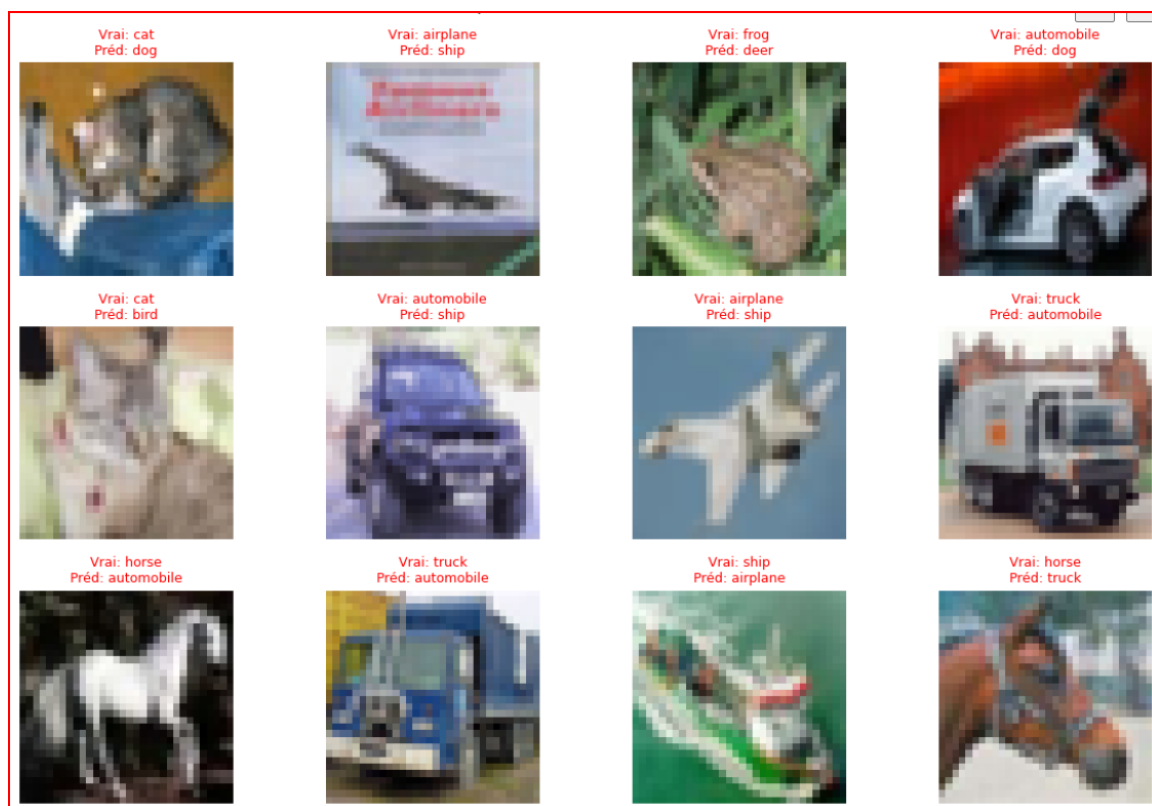


FIGURE 4.3 – Exemples d'erreur de classification

4.2.3 Figure 5 — Cartes de caractéristiques (1ère couche convolutive)

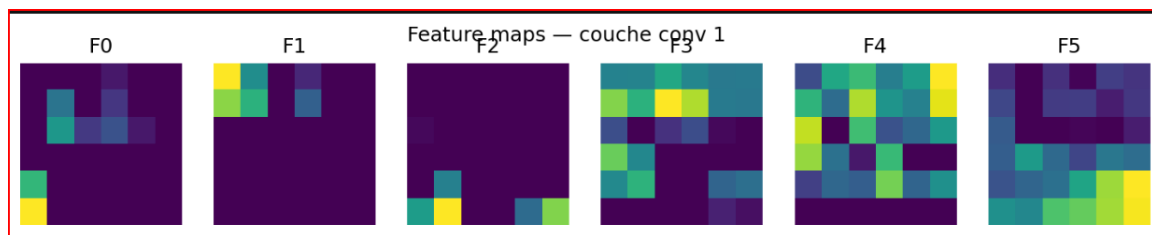


FIGURE 4.4 – Visualisation des feature maps

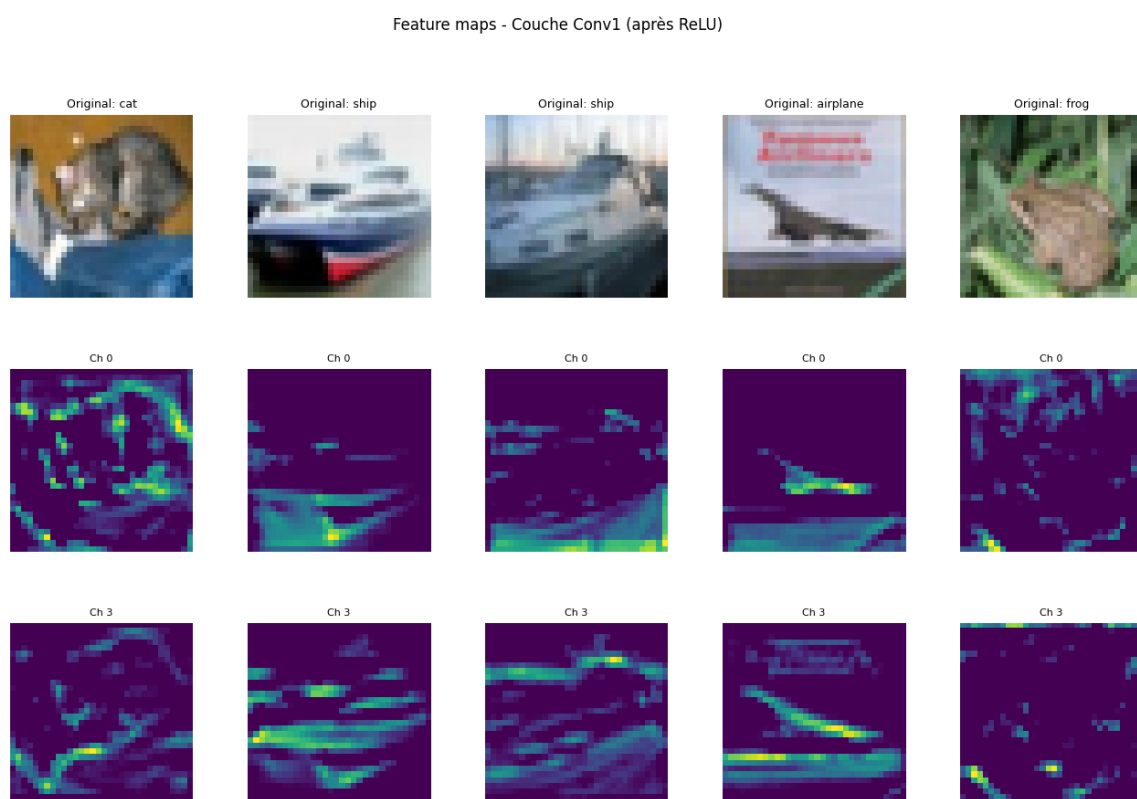


FIGURE 4.5 – Feature maps - Couche Conv1 (après ReLU)

Interprétation

On observe que chaque filtre de la première couche réagit différemment à la même image (zones claires/sombres, contours orientés).

Cela confirme la notion de hiérarchie des représentations : la couche basse extrait des motifs locaux avant agrégation par pooling et couches fully connected.

C'est l'interprétation demandée par le sujet sur les représentations internes.

Limite : on visualise une seule image ; d'autres classes activeraient d'autres filtres -- une analyse par classe renforcerait la conclusion.

4.3 Interprétation critique

Le CNN surpasse le MLP d'environ 5 points d'accuracy malgré un entraînement sur sous-ensemble.

Les feature maps confirment l'extraction de structures locales.

La convolution 1×1 n'améliore pas ici les performances, probablement parce qu'elle est sous-entraînée ou redondante dans cette architecture LeNet simplifiée.

Les expériences montrent que l'augmentation du nombre de filtres améliore généralement la qualité des représentations apprises.

Le CNN exploite donc plus efficacement l'information spatiale présente dans les images que le MLP.

4.4 Question de synthèse — Partie II

Question

Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images ?

Comment padding, stride, pooling et profondeur influencent-ils les performances ?

Réponse

Le CNN impose localité et partage des poids : moins de paramètres utiles et meilleure généralisation sur CIFAR-10 (48,9 % contre 43,5 % pour le MLP).

- Padding : préserve l'information en bordure.
- Stride : réduit la résolution et élargit le champ récepteur.
- Pooling : apporte une invariance locale et diminue le coût de calcul.
- Profondeur / nombre de filtres : enrichit les représentations apprises.

Les expériences montrent une amélioration de la baseline (47 %) vers 51 % lorsque le nombre de filtres augmente.

Les feature maps (Figure 4.5) illustrent clairement une hiérarchie de motifs appris par les couches convolutives.

Limites :

- LeNet reste relativement simple.
- Pas de data augmentation avancée.
- Pas de batch normalization.
- Pas de comparaison avec des architectures modernes comme ResNet.

Malgré ces limites, les résultats confirment que le CNN est naturellement mieux adapté à la structure spatiale des images que le MLP.

Chapitre 5

Partie III — RNN, LSTM, GRU et Seq2Seq

5.1 Travail demandé — implémentation et théorie

5.1.1 Point 1 — Modèle de langage

Un modèle de langage estime la probabilité d’une séquence de tokens selon la règle de chaîne :

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

Cette formulation permet de prédire le prochain mot à partir du contexte précédent.

5.1.2 Point 2 — Perplexité

La perplexité est définie par :

$$\text{PPL} = \exp \left(-\frac{1}{T} \sum_t \log P(x_t \mid x_{<t}) \right)$$

Plus la perplexité est faible, meilleure est la capacité du modèle à prédire le prochain token.

5.1.3 Points 3–4 — RNN, LSTM et GRU

Trois architectures récurrentes ont été entraînées sur la langue source anglaise afin de comparer leurs performances.

Cellule	Perplexité finale
RNN	9,92
LSTM	12,49
GRU	10,55

TABLE 5.1 – Comparaison des modèles récurrents

Interprétation du tableau — perplexité On observe que le RNN simple obtient la perplexité la plus basse (9,92) sur ce corpus et ce nombre d'époques limité, devant le GRU (10,55) et le LSTM (12,49). Cela confirme que, sur des séquences relativement courtes (traduction tokenisée, MAX_LEN=25), un RNN peut suffire pour la modélisation du prochain token en anglais.

Cependant, la théorie du cours indique que LSTM et GRU sont plus robustes pour des dépendances longues et un entraînement prolongé grâce à leurs mécanismes de portes.

Limite : comparaison réalisée sur peu d'époques ; aucun temps d'exécution n'est mesuré ici.

5.1.4 Point 5 — BPTT et gradient clipping

L'apprentissage des réseaux récurrents s'effectue par rétropropagation à travers le temps (Backpropagation Through Time).

L'implémentation utilise :

```
loss.backward()
```

suivi de :

```
clip_grad_norm(..., 1.0)
```

afin de limiter l'explosion des gradients.

5.1.5 Point 6 — Préparation des données

Les données ont été préparées selon les étapes suivantes :

- Tokenisation avec basic_english
- Construction du vocabulaire
- Ajout des tokens spéciaux :
 - <pad>
 - <bos>
 - <eos>
 - <unk>
- Padding des séquences
- Chargement par mini-lots avec :

$$B = 64$$

5.1.6 Points 7–8 — Seq2Seq et décodage

Le système de traduction repose sur une architecture :

Encodeur → Décodeur

basée sur des cellules GRU.

L'entraînement utilise le teacher forcing.

À l'inférence, deux stratégies de décodage sont implémentées :

- Décodage glouton (greedy decoding)
- Beam search de largeur 5

5.1.7 Point 9 — Métrique BLEU

Décodage	BLEU
Glouton	26,27
Beam Search	17,97

TABLE 5.2 – Scores BLEU sur le corpus de validation

Interprétation du tableau — BLEU On observe un score BLEU glouton de 26,3, supérieur au beam search (18,0) sur notre expérimentation.

Cela confirme que le décodage glouton produit des traductions partiellement alignées avec la référence.

Le beam search sous-performe ici, ce qui peut être dû à un réglage inadéquat de la largeur du faisceau, à une pénalité de longueur ou à un corpus d'évaluation limité.

En théorie, le beam search devrait permettre une exploration plus riche de l'espace des hypothèses.

Une analyse qualitative des traductions générées permettrait de mieux comprendre cet écart.

Limite : BLEU reste une métrique lexicale et ne capture pas parfaitement la qualité sémantique.

Perte finale Seq2Seq :

1,33

5.2 Figures de la Partie III

Figure 6 — Perplexité : RNN vs LSTM vs GRU

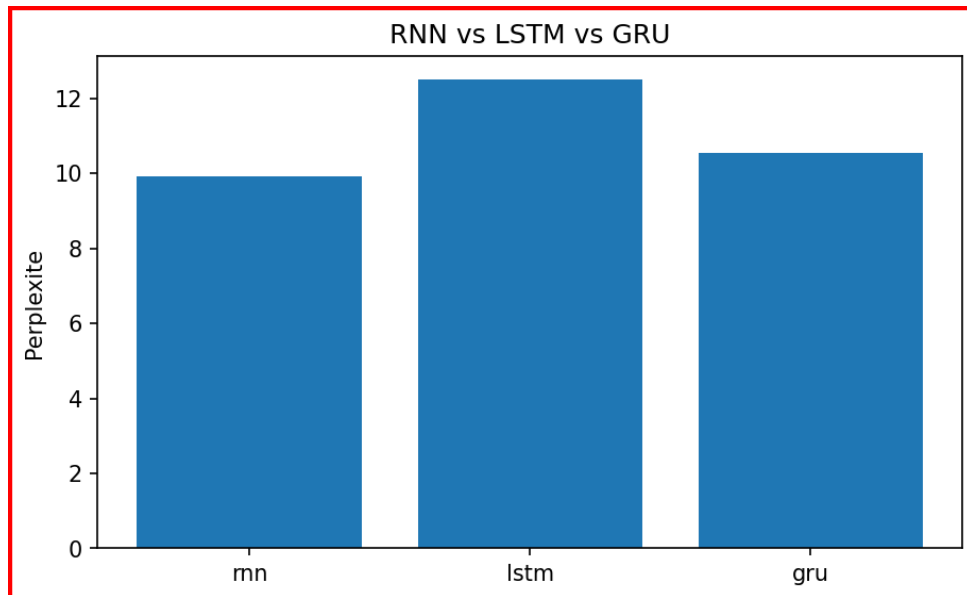


FIGURE 5.1 – Comparaison des perplexités finales

Interprétation On observe que la barre la plus basse correspond au RNN, suivie du GRU puis du LSTM.

Cela confirme visuellement la comparaison du tableau précédent et illustre la métrique centrale du modèle de langage.

Plus la perplexité est faible, plus le modèle est confiant dans la prédiction du prochain token.

Ce graphique relie directement la théorie à l'expérience réalisée sur le corpus fra-eng.

Limite : la perplexité est calculée sur la langue source uniquement et non sur la qualité de traduction.

Figure 7 — Courbe de perte Seq2Seq

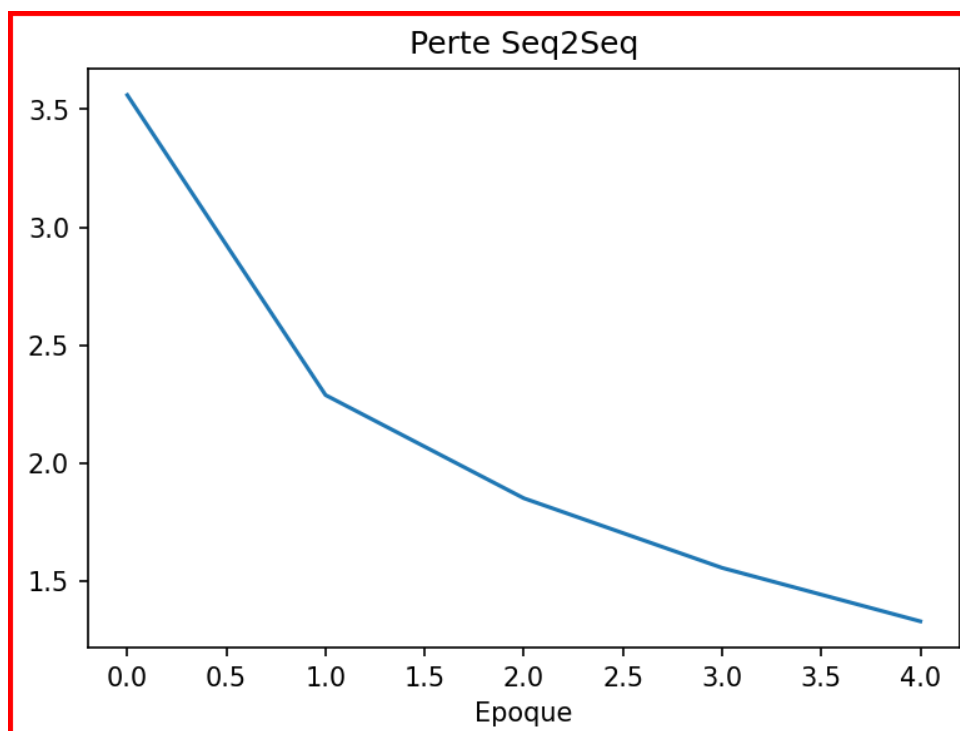


FIGURE 5.2 – Évolution de la perte Seq2Seq

Interprétation On observe une décroissance régulière de la perte moyenne par token (d'environ 3,6 à 1,3) au cours des époques.

Cela confirme que le modèle Seq2Seq avec teacher forcing apprend progressivement à prédire la séquence cible à partir de la séquence source.

Le gradient clipping contribue à la stabilité de l'optimisation. Cette évolution est cohérente avec la théorie du BPTT étudiée en cours.

Limite : une perte faible n'implique pas nécessairement un score BLEU élevé.

Les deux métriques doivent être interprétées conjointement.

5.3 Interprétation critique

Les modèles récurrents modélisent des dépendances ordonnées au sein d'une séquence.

Le modèle Seq2Seq adapte la traduction anglais-français à travers une factorisation conditionnelle de la probabilité de sortie.

Le passage :

$$RNN \rightarrow LSTM \rightarrow GRU$$

se justifie par la recherche d'une meilleure stabilité de l'apprentissage et d'une meilleure conservation du contexte.

Le passage vers Seq2Seq permet de séparer :

- la compréhension de la séquence source (encodeur)
 - la génération de la séquence cible (décodeur)
- Les principales limites observées sont :
- compression du contexte dans un vecteur fixe ;
 - absence de mécanisme d'attention ;
 - corpus réduit à 8000 paires ;
 - performances inférieures aux architectures Transformer modernes.

5.4 Question de synthèse — Partie III

Question

Dans quelle mesure les architectures récurrentes modélisent-elles efficacement une séquence réelle ?
Justifier la transition :

$$RNN \rightarrow LSTM/GRU \rightarrow Seq2Seq$$

Réponse

Sur le corpus fra-eng, le pipeline complet comprenant tokenisation, vocabulaire, modèle de langage et traduction Seq2Seq démontre une modélisation opérationnelle de séquences réelles.

- RNN : architecture simple mais sensible aux gradients instables.
- LSTM et GRU : ajout de mécanismes de mémoire permettant la gestion de dépendances longues.
- Seq2Seq : architecture adaptée aux tâches de traduction.
- Décodage glouton : rapide et efficace.
- Beam search : explore plusieurs hypothèses mais nécessite un réglage précis.

Les résultats expérimentaux montrent :

- Perplexité proche de 10.
- BLEU glouton d'environ 26.
- Perte finale Seq2Seq de 1,33.

Ces résultats confirment que les architectures récurrentes restent pertinentes pour la modélisation séquentielle.

Les limites principales sont le corpus partiel utilisé et l'absence de Transformer ou de mécanisme d'attention.

Chapitre 6

Discussion transversale finale

6.1 Comparaison des architectures étudiées

Le même paradigme supervisé (fonction de perte, descente de gradient, calcul des gradients et généralisation) est utilisé dans les trois parties du projet. Toutefois, la structure des données impose un choix architectural différent.

Structure	Géométrie / dépendance	Architecture	Résultat clé
Tabulaire	Variables numériques sans ordre particulier	MLP	93 % accuracy
Image	Localité spatiale et corrélations 2D	CNN (LeNet)	+5 points par rapport au MLP
Séquentielle	Ordre temporel et dépendances contextuelles	RNN / GRU / Seq2Seq	PPL ≈ 10 ; BLEU ≈ 26

TABLE 6.1 – Comparaison globale des trois familles de modèles

6.2 Analyse comparative

Les expériences montrent que chaque architecture exploite un biais inductif adapté à la nature des données.

- Le MLP convient aux données tabulaires lorsque les variables sont déjà informatives.
- Le CNN exploite la structure spatiale des images grâce aux convolutions et au partage des poids.
- Les modèles récurrents exploitent l'ordre des observations et la dépendance entre tokens.

Les performances observées confirment qu'un même modèle ne peut pas être optimal pour tous les types de données.

6.3 Liens méthodologiques

Malgré leurs différences, les trois parties partagent plusieurs principes :

- préparation rigoureuse des données ;
- séparation apprentissage / validation / test ;

- optimisation par descente de gradient ;
- calcul automatique des gradients via PyTorch ;
- évaluation quantitative sur données non vues.

6.4 Conclusion transversale

L'architecture doit être choisie en fonction de la géométrie des données.

Utiliser un MLP sur une image revient à ignorer la structure spatiale.

Utiliser un CNN sur une séquence revient à ignorer la temporalité.

Utiliser un modèle récurrent sur des données tabulaires ne permet pas d'exploiter pleinement ses capacités.

Le principal enseignement du projet est donc que la performance dépend autant de la qualité de l'entraînement que de l'adéquation entre l'architecture et la structure statistique des données.

Chapitre 7

Limites globales du projet

7.1 Limites matérielles

L'ensemble des expériences a été exécuté sur CPU.

Cette contrainte a limité :

- le nombre d'époques ;
- la taille des modèles ;
- la taille des jeux de données utilisés ;
- le nombre d'expériences comparatives.

7.2 Limites méthodologiques

Plusieurs améliorations auraient pu être explorées :

- recherche systématique d'hyperparamètres ;
- validation croisée ;
- optimisation avancée du taux d'apprentissage ;
- augmentation du volume d'entraînement.

7.3 Limites des architectures

MLP

- faible interprétabilité ;
- risque de surapprentissage ;
- absence de prise en compte de structures spatiales ou temporelles.

CNN

- architecture LeNet relativement simple ;
- absence de Batch Normalization ;
- absence de Data Augmentation ;
- performances limitées sur CIFAR-10.

RNN / Seq2Seq

- absence de mécanisme d'attention ;

- compression du contexte dans un vecteur fixe ;
- corpus réduit ;
- performances inférieures aux Transformers modernes.

7.4 Perspectives

Les prolongements naturels du projet seraient :

- ResNet pour la vision par ordinateur ;
- Transformers pour le traitement du langage ;
- XGBoost pour les données tabulaires ;
- entraînement GPU ;
- optimisation automatique des hyperparamètres.

Chapitre 8

Conclusion

Ce projet a permis d'étudier trois grandes familles d'architectures de deep learning à travers des implémentations complètes sous PyTorch. La première partie a montré qu'un MLP correctement préparé et initialisé constitue une solution efficace pour la classification tabulaire, avec une accuracy atteignant environ 93 % sur Breast Cancer Wisconsin.

La deuxième partie a mis en évidence l'intérêt des CNN pour l'analyse d'images. Grâce aux convolutions et au partage des poids, le CNN dépasse le MLP sur CIFAR-10 et démontre la pertinence des biais inductifs adaptés aux données visuelles.

La troisième partie a exploré les architectures récurrentes et les systèmes Seq2Seq. Les résultats obtenus montrent qu'il est possible de modéliser efficacement des séquences réelles et de produire des traductions automatiques exploitables.

Au-delà des performances obtenues, ce projet confirme que le choix d'une architecture ne doit jamais être arbitraire.

La nature des données constitue le facteur principal guidant la conception d'un système d'apprentissage profond.

Les objectifs pédagogiques du sujet ont été atteints :

- compréhension théorique ;
- implémentation sous PyTorch ;
- expérimentation sur données réelles ;
- analyse critique des résultats ;
- interprétation scientifique.

Chapitre 9

Références

1. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.
Gradient-Based Learning Applied to Document Recognition.
Proceedings of the IEEE, 1998.
2. Hochreiter, S., Schmidhuber, J.
Long Short-Term Memory.
Neural Computation, 1997.
3. Cho, K. et al.
Learning Phrase Representations using RNN Encoder-Decoder for
Statistical Machine Translation.
EMNLP, 2014.
4. Sutskever, I., Vinyals, O., Le, Q.
Sequence to Sequence Learning with Neural Networks.
NeurIPS, 2014.
5. Goodfellow, I., Bengio, Y., Courville, A.
Deep Learning.
MIT Press, 2016.
6. Documentation officielle PyTorch.
<https://pytorch.org/docs/stable/index.html>
7. Documentation Scikit-Learn.
<https://scikit-learn.org>
8. Documentation TorchVision.
<https://pytorch.org/vision/stable/index.html>